

TMF630 REST API Design Guidelines Part 8: TM Forum API Specialization for a context

TMF630 Part 8

Maturity Level: General Availability (GA)	Team Approved Date: 12-Mar-2026
Release Status: Pre-production	Approval status: Team Approved
Version: 5.0.0	IPR Mode: RAND

Notice

Copyright © TM Forum 2025. All Rights Reserved.

This document and translations of it may be copied and furnished to others, and derivative works that comment on or otherwise explain it or assist in its implementation may be prepared, copied, published, and distributed, in whole or in part, without restriction of any kind, provided that the above copyright notice and this section are included on all such copies and derivative works. However, this document itself may not be modified in any way, including by removing the copyright notice or references to TM FORUM, except as needed for the purpose of developing any document or deliverable produced by a TM FORUM Collaboration Project Team (in which case the rules applicable to copyrights, as set forth in the [TM FORUM IPR Policy](#), must be followed) or as required to translate it into languages other than English.

The limited permissions granted above are perpetual and will not be revoked by TM FORUM or its successors or assigns.

This document and the information contained herein is provided on an “AS IS” basis and TM FORUM DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

Direct inquiries to the TM Forum office:

100 Enterprise Drive
Suite 301 #1649
Rockaway, NJ 070866, USA
Tel No. +1 862 227 1648

TM Forum Web Page: www.tmforum.org

Table of Contents

Notice	2
Executive Summary	4
Conventions	5
Key words	5
Definitions	5
Acronyms	5
Objective	6
Principles	8
Rules	11
Rules for officialy approved TM Forum Specialized API	16
References	18
Administrative Appendix	19
Document History	19
Acknowledgments	19

Executive Summary

The “REST API Design Guidelines” document provides guidelines and design patterns used in designing TM Forum REST APIs. The document is organized in multiple parts as follows:

- **Part One:** Practical guidelines for RESTful APIs; naming, CRUD, filtering, tasks, history, notifications
- **Part Two:** Advanced guidelines for RESTful API polymorphism, extension patterns, depth and expand directive, entity RefOrValue
- **Part Three:** Guidelines for extending TMF Open APIs with hypermedia support
- **Part Four:** Advanced guidelines for RESTful APIs; lifecycle management, common tasks
- **Part Five:** JSON Patch extension to manage arrays
- **Part Six:** JSON Path extension
- **Part Seven:** JSON Schema patterns
- **Part Eight:** API Specialization

This part (Part 8) describes how to specialize a TM Forum API respecting TM Forum guidelines. Our objective is to guide developers and architects when there is a need to specialize a TM Forum API.

API Specialization is a methodology and toolset to take an existing TM Forum API and adjust it to a well-defined business need, typically by removing any unnecessary properties or operations that are not needed for the use case. Additional operations and properties may also be included to make a concrete interface for efficient integrations.

Conventions

Key words

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 8174 [RFC8174].

Definitions

- **Base API** : a standard TMF API, unspecialized. For instance TMF622 Product Ordering Management.
- **Specialized API** : an API that is a specialization of another TMF API, either a Base API or another Specialised API. For instance TMF622_CA Context A Product Ordering Management. The Specialized API follows all the rules and principles given in these guidelines.
- **Parent API** : how to refer to the (Base or Specialized) API that a Specialized API is a specialisation of. The Parent API has no awareness of its children. For instance TMF622 Product Ordering Management is the Parent API (and the Base API) of TMF622_CA Context A Product Ordering Management. The Specialized API follows all the rules and principles given in these guidelines.
- **Operation** : an operation is defined by a REST verb (GET, POST, PATCH, DELETE) and the endpoint (REST resource) it applies to. TBD ADD EXAMPLE

Acronyms

- **ABE** : Aggregated Business Entity
- **API** : Application Programming Interface. In this document an "API" means the specification (OAS, documentation, etc.), not the implemented API itself.
- **CA** and **CB** : Context A and Context B, for sake of examples
- **CTK** : Conformance Test Kit
- **DCS** : Domain Context Specialization was how TM Forum originally referred to the need to create more concrete versions of our catalog-driven, generic APIs. TM Forum no longer use this terminology or the original approach involving core schemas.
- **OAS** : OpenAPI Specification
- **ODA** : Open Digital Architecture
- **SDO** : Standard Organization

Objective

Standard TM Forum API specifications provide a generic representation of interface between applications, based on other TM Forum assets (Information Framework, eTOM, ODA components and End to end Use Cases) and providing a relevant set of assets for each API. They are designed to support generic business processes and are not tailored to specific use cases. Most of the attributes are optional and non-constrained (e.g. by enum). This approach may lead to low interoperability between components because TMF API specifications are open to different interpretations for similar use cases. This is in contrast with tightly bound protocols (such as in a communications network) that leave no room for such different interpretation.

TM Forum API specialization aims to provide a mechanism to allow a tighter contract in contexts that require a strong interoperability between API consumer and provider. This permits an increase to interoperability by giving to developers of API Client and API Server an accurate and precise specification applicable for the context.

Illustrations :

- Given a requirement to add some data to the representation of a Geographic Site, such as the number of employees working in the site. If the standard TMF API does not provide such an attribute, it is possible to add a new attribute such as `numberOfEmployees` for this context. If this need seems to be valuable for most contexts then it can be provided as a standard attribute in the base TMF API; if not, then each context that needs this data will have to provide its own attribute for this need.
- The representation of a Set-top Box is not currently defined in any TMF API, but could be defined as a specialisation of a Physical Resource.
- TMFC002 Product Order Capture and Validation ODA component exposes only 1 single TMF622 Product Order API that should cover all use cases managed by TMFC002 component. But a context might need to provide an accurate representation of the strictly needed attributes and operations for their use case.

Note that there is not a single way to specialize the same API for the same requirement.

- Specialization can be done internally in an Operator IT department, without needing to standardize those specializations.
- Some contexts that require interaction between several Organizations need to standardize these API specialization specifications. Standardization can be done under the auspices of TM Forum or another standards organization (Mplify, GSMA, etc.). Depending on the context, there may be a requirement for the specialized API to be an

official TM Forum API available in the TM Forum API directory.

Nevertheless the specialization should remain as much as possible compatible with Base API specifications and limited in number, in order to avoid explosion of specializations of same API for each use case. Such an explosion would contradict the objective of interoperability.

This document aims to give the best possible understanding on how all these TM Forum assets (base TMF Open API, Specialized TMF Open API, ODA components) are always consistent.

These guidelines don't describe the "How To" of API Specialization; that information belongs in IG1353 TM Forum API Developer's Handbook.

Principles

This part lists principles that are the foundation of the specialization guidelines.

Principle 1 : A Specialized API is a specialization of a single TM Forum Base API for a specific context.

A Specialized API can exist only if there is a Base API to be specialized. So the Base API to be specialized has to exist and be stable before the specialization is done.

The Specialized API can be a direct specialization of the Base API or an indirect (ie the Parent API is not a Base API). No composite specializations are supported as part of the specialization guidelines.

Principle 2 : A Specialized API is a complement to a Base API, not a replacement.

A Base Open API SHOULD cover generic business processes that can be used for any use case. The TM Forum encourages best compatibility by making sure the Base TMF API allows many use cases in order to reduce complexity and focus efforts on a limited number of assets.

For instance the same implementation of a TMF622 Product Order API could be used for two different contexts.

There SHOULD be no need to specialize a TMF API for a particular context if this API already exists in the Operator IT.

Specialization can be an initial step for a context, so MAY later become obsolete with the changes included in the Base API. For instance to define and expose a composite ProductOffering that bundles a Context A ProductOffering and a Context B ProductOffering, it MAY be needed to use the standard TMF620 instead of one of the TMF620 Specialized APIs.

Principle 3 : Business requirements are the driver for creating a Specialized API.

A context that asks for specialization can be a wide ecosystem implying multiple API to be specialized, describing end-to-end process for a set of consistent business requirements. The business requirements can come from outside TM Forum, such as from the GSMA Open Gateway initiative that drives TM Forum Open Gateway Operate API requirements.

Principle 4 : Specialization of multiple APIs MAY be designed and used together.

A business context might require a set of Specialized API from a set of Base TM Forum APIs. Each specialization is separate but MAY re-use transforms to produce a coherent whole.

Principle 5 : API Specialization doesn't imply specialization in other TM Forum domains.

The Specialized API specificities MAY not be of interest to be standardized in common TM Forum assets like Information Framework, eTOM or Functional Framework, except if this specialization needs a feature that is missing in those domains and that could be beneficial for other contexts.

Principle 6 : A Specialized API MAY be hosted by TM Forum as part of the API resources for the Parent API.

Each TMF member can create its own specialization without being under TM Forum governance. Where the context implies benefits from sharing the standard between organizations, TM Forum can host and maintain this specialization. Other SDOs can also build on this approach.

Principle 7 : A Parent API MAY continue to evolve regardless of their Specialized API.

A parent API has no awareness of its children.

Principle 8 : A Specialized API MIGHT NOT follow its Parent API lifecycle.

A Specialized API is based on an Parent API version. Specialized APIs can upgrade at a time that suits them. The Specialized API CAN be updated to take the Parent API updated version at any time once the Parent API is stable.

Just like any other TMF API, the specialized APIs MAY be deprecated if they are no longer in use or do not have an owner.

Principle 9: Specialized API may use specification-centric patterns for modeling specialized concepts.

Specialized API may use characteristic-based extension patterns if available in the Parent API for a given concept. For example a specialization of TMF622 can use common TMF characteristic patterns in combination with productSpecification defined in TMF620.

In other words it SHOULD be possible to choose between both options for a Specialized API :

- characteristic-based extension pattern : keeping common characteristics name/value pairs as defined in the Base API
- schema-based extension pattern (strongly-typed schemas) as described in TMF630 REST Guidelines chapter 2, in particular with targetEntity schemas defined in the EntitySpecification schemas that permits to replace the generic characteristics name/value pairs

Principle 10 : A Specialized API MAY require a specific conformance profile and a specific CTK.

By default, a Specialized API is bound to its Parent API Conformance Profile and CTK, so it does not need a specific Conformance Profile and specific CTK. Some contexts MAY require a specific conformance profile and CTK to check that the Specialized API is implemented as defined by the specific conformance profile. In that case the specialized conformance profile and specialized CTK are a specialization of the Parent API conformance profile and Parent CTK adding more constraints for the context.

Rules

This part lists rules based on above principles.

Rule 1 : Specialization SHOULD be avoided when additional documentation for the context is enough

When the API client consumes an API only for a specific context, Specialization can help to make a minimal implementation that is complete for the purpose at hand.

But before specializing an API, a study SHOULD be done to check if using standard Base TMF API and contextualized documentation for the developers would be enough.

Rule 2 : Extensions made in a Specialized API SHOULD be proposed for inclusion in the Parent API if they have wider applicability and if they are useful in another specialised context.

The goal is to limit the need for specialization and to increase consistency between Parent API and Specialized API. This rule permits quality improvements to a Parent API and for other Specialized APIs to benefit from the new Parent features.

All extensions MUST be traced and made visible, so that other specializers can reuse.

Rule 3 : A Specialized API MUST keep all required attributes from the Parent API.

Required attributes are defined in OAS. For the operations that are specialized from the Parent API, the data model definition for request/response MUST be compatible with parent API. In particular all the required fields MUST be preserved in the specialized models.

Rule 4 : A Specialized API implementation MUST pass its Parent API CTK.

So by consequence it MUST pass also the Base API CTK that it specializes.

NOTE : Currently the result of an API certification by a CTK is binary : "passed" or "failed". There is no possibility to declare the operations that are to be formally passed by CTK. So for a Specialized API that doesn't expose some required operations of Parent API, the Specialized API doesn't pass officially the CTK. TM Forum has in its roadmap to address this concern, and give a solution to allow the Specialized API to officially pass the Parent CTK based on the operations it specializes from the Parent API. This allows the Specialized API to pass Parent API CTK even if a subset of the Parent API required operations are implemented.

Rule 5 : A Specialized API MAY reduce its Parent API.

The reduction can be done by :

- omit an operation from the Parent API (including a required operation). Note that omitting all operations for the same endpoint implies the omission of the whole endpoint.
- omit an optional attribute from the Parent API
- restrict an open string attribute in the Parent API by using an enumeration (enum, const)
- restrict an open string attribute in the Parent API by using a format (date-time...)
- constraint an attribute in the Parent API by using a validator (minLength, maxLength, pattern, minItems, maxItems...)

When a parent API attribute is defined as an enum, it is not allowed to remove values from this list.

Rule 6 : A Specialized API MAY extend its Parent API.

The Specialized API MUST be backward compatible with its Parent API, but the reverse need not be true.

Extension in the Specialized API can be done by :

- make an optional attribute of the Parent API required
- add a new (optional or required) attribute
- add a new endpoint
- add a new operation for an existing endpoint
- overwrite the description of an attribute
- replace the examples of the Parent API

When a parent API attribute is defined as an enum, it is not allowed to add values to this list. This can be done by adding a new attribute for this purpose.

In order to avoid proliferation of endpoints and discrepancy over the time with the other Specialized API from the same Parent API, adding new endpoints and operations to the parent API SHOULD be carefully studied.

Rule 7 : A Specialized API MUST NOT change the endpoint naming of the parent API.

This permits the API client to keep calling same endpoints as the Parent API. The use of polymorphism with @type permits to precise the context and so the specialized implementation of the Parent API.

Rule 8 : A Specialized API MUST NOT redefine an existing attribute from its Parent API (keeping same attribute name) for a different functional meaning.

The functional meaning of an attribute SHOULD be defined by an Information Framework entity (ABE) if applicable.

But a Specialized API MAY redefine an existing attribute from its Parent API (keeping same attribute name) for same functional meaning.

For instance the description of the attribute, an addition of a format constraint are allowed. But this has to be limited and controlled to avoid gaps between Specialized API its Parent API. The general rule is that the Specialized API reuses as-is the selected attributes of its Parent API.

Rule 9 : A Specialized API MUST NOT change the type of an existing attribute in a non-backward compatible way from the Parent API attribute type.

For instance it's not possible to change an attribute type from string to integer.

Rule 10 : A Specialized API MAY change the referred schema of an existing attribute from its Parent API.

This rule can be applied only if all the required attributes of at least one of the referred schema are kept required.

- Extending referred schema

Example of "itemPrice" attribute of ProductOrderItem schema from Base API TMF622 Product Ordering Management. In the specialized API OAS, itemPrice refers to OrderPrice schema like in the Parent API.

```
itemPrice:
  type: array
  items:
    $ref: '#/components/schemas/OrderPrice'
```

In addition, OrderPrice schema adds ContextA schema in its discriminator list.

```
OrderPrice:
  allOf:
    - $ref: '#/components/schemas/Extensible'
    - type: object
  description: >-
```

```

        An amount, usually of money, that represents the actual price paid by
        the Customer for
        this item or this order
        properties:

```

```

...

```

```

discriminator:
  propertyName: '@type'
mapping:
  OrderPrice: '#/components/schemas/OrderPrice'
  ContextAOrderPrice: '#/components/schemas/ContextAOrderPrice'

```

ContextAOrderPrice schema contains an allOf OrderPrice schema.

```

ContextAOrderPrice:
  allOf:
    - $ref: '#/components/schemas/OrderPrice'
    - type: object
      description: >-
        Specific description for ContextA OrderPrice
      properties:
        additionalAttribute:
          type: string
          description: Description of the additional attribute for
contextAOrderPrice

```

- Reducing referred schema

In that case nothing changes in the OAS except that the attributes omitted from a Parent schema are no more in the schema.

- reducing oneOf list

Example of product attribute of ProductOrderItem schema in Base API TMF622 Product Ordering Management :

```

product:
  $ref: '#/components/schemas/ProductRefOrValue'

```

ProductRefOrValue schema being in the Parent API :

```

ProductRefOrValue:

```

```

type: object
description: >-
  The polymorphic attributes @type, @schemaLocation & @referredType are
  related to the Product
  entity and not the ProductRefOrValue class itself
oneOf:
  - $ref: '#/components/schemas/Product'
  - $ref: '#/components/schemas/ProductRef'
discriminator:
  propertyName: '@type'
  mapping:
    Product: '#/components/schemas/Product'
    ProductRef: '#/components/schemas/ProductRef'

```

A Specialized API can reduce the oneOf list, removing one or several referred schemas from the list. In the Specialized OAS "product" attribute remains unchanged.

```

product:
  $ref: '#/components/schemas/ProductRefOrValue'

```

But ProductRefOrValue contains only the remaining schemas.

```

ProductRefOrValue:
  type: object
  description: >-
    The polymorphic attributes @type, @schemaLocation & @referredType are
    related to the Product
    entity and not the ProductRefOrValue class itself
  oneOf:
    - $ref: '#/components/schemas/ProductRef'
  discriminator:
    propertyName: '@type'
    mapping:
      ProductRef: '#/components/schemas/ProductRef'

```

- extending oneOf list

For extension the mechanism is the same as for reduction except that the list of referred schemas in the oneOf list contain additional schemas compared to the list in the Parent API.

Rules for officialy approved TM Forum Specialized API

This chapter describes the rules to identify the Specialized API and schemas that are managed under TM Forum or other SDO auspices. So this paragraph is applicable to Specialized API aiming to be an official TM Forum asset published within TM Forum website.

Rule OA1 : A Specialized API MUST take into account the main TM Forum frameworks (ODA, Information Framework...).

The idea is that the Specialized API MUST reuse existing concepts, entities, schemas rather than recreating the wheel.

Rule OA2 : A Specialized API MUST MUST have his API Profile document approved before being specified.

The main goal of the API Profile is to document and justify all changes from the Parent API (reduction, extension...).

Rule OA3 : A Specialized API MUST be compliant with TMF630 Design Guidelines and MUST be approved by TM Forum stakeholders.

Rule OA4 : A Specialized API MUST be generated using the TM Forum tooling if it is meant to be a standard TM Forum API.

A specialization of a TMF API that is not aimed to be a TM Forum standard can be done without the TMF tooling.

Rule OA5 : The context name and prefix MUST be officially approved by TM Forum.

For instance the Wholesale Broadband program uses "WB" as its prefix.

Rule OA6 : A Specialized API TMF identifier number MUST keep the Base API identification number.

The Base API number is kept for all API that specializes this Base API, including Specialized of Specialized. That permits to be quickly aware of the Base API being specialized. For instance a Specialized API of TMF622 Product Ordering Management MUST have an identifier that starts with TMF622. NOTE : This identification applies only if the Specialized API follows the rules and principles in these guidelines; in particular rule 4 "A Specialized API MUST pass its Parent API CTK by including all required aspects.". Any API that specialises without applying rule 4 MUST have a specific number.

Rule OA7 : A Specialized API TMF identifier MUST extend the Base API identifier with an underscore followed by the upper case acronym of the context.

This permits to distinguish the identifier of the Specialized API from its Base API identifier.

For instance a Specialized API of TMF622 Product Ordering Management within context A MUST have the identifier TMF622_CA. And a Specialized API of TMF622_CA within context B MUST have the identifier TMF622_CB. For Wholesale Broadband program the TMF API identifier must be TMF622_WB.

Note that the Parent API identifier is present in the OAS. So in case the Parent API is not the Base API, the trace of the Parent API is given in the OAS of the Specialized API.

Rule OA8 : A Specialized API TMF name MUST extend the Base API name adding the full name of the context after the Base API name.

For instance a Specialized API of TMF622 Product Ordering Management within context A MUST have the name "Product Ordering Management Context A". And a Specialized API of TMF622_CA within context B MUST have the name "Product Ordering Management Context B". For Wholesale Broadband program the TMF API name must be "Product Ordering Management Wholesale Broadband".

Rule OA9 : The name of a Specialized schema MUST extend its Base schema name adding the unique prefix allocated to the context before the schema name.

Each context has a unique allocated prefix by TM Forum. The preferred prefix is the UpperCamelCase (also known as PascalCase) full name of the context, but acronym or simplified prefix is allowed. For instance a Specialized schema name of ProductOrder within context A should have the name "ContextAProductOrder" or some form of a unique allocated prefix. Another example : TM Forum allocated the **WsBb** prefix for WholesaleBroadband thus the schema name must be "WsBbProductOrder" under this context. Note that currently the schema name is used as the @type discriminator value in the TM Forum Open API OAS.

References

- [SID]
TMForum Information Framework
- [TMF630-1]
TMForum
REST API Design Guidelines
Part 1: Practical guidelines for RESTful APIs naming, CRUD, filtering, tasks, history, notifications
- [TMF630-2]
TMForum
REST API Design Guidelines
Part 2: Advanced guidelines for RESTful APIs polymorphism, extension patterns, depth and expand directive, entity RefOrValue
- [TMF630-3]
TMForum
REST API Design Guidelines
Part 3: Guidelines for extending TMF Open API's with hypermedia support
- [TMF630-4]
TMForum
REST API Design Guidelines
Part 4: Advanced guidelines for import and export tasks and for entity versioning
- [TMF630-5]
TMForum
REST API Design Guidelines
Part 5: JSON patch extensions to manage arrays
- [TMF630-6]
TMForum
REST API Design Guidelines
Part 6: JSON path extensions
- [TMF630-7]
TMForum
REST API Design Guidelines
Part 7: JSON Schemas guideline

Administrative Appendix

This Appendix provides additional background material about the TM Forum and this document. In general, sections may be included or omitted as desired, however a Document History must always be included.

Document History

Version History

This section records the changes between this and the previous document version as it is edited by the team concerned.



this is an incremental number which does not have to match the release number and used for change control purposes only.

Version Number	Date Modified	Modified by	Description of changes
5.0.0-beta1	31 October 2025	Olivier Arnaud based on all contributors inputs	Initial version based on 2025 working group study
5.0.0-beta2	09 December 2025	Olivier Arnaud	Updates from London December 2025 SpecJam (principles, rules, chapters)
5.0.0	12 March 2026	Olivier Arnaud	Feedback from Architecture group and Wholesale Broadband group

Acknowledgments

Contributors to Current Version

The following members of the TM Forum API team contributed to this version of the document (in alphabetical order):

Bartosz Michalik	Amartus	Contributor
Joel Burgess	Oracle	Contributor
Lorna Mitchell	TM Forum	Contributor, Editor
Olivier Arnaud	Orange	Contributor, Editor

Patrick Farkas	Vodafone	Reviewer
Pierre Close	ATT	Contributor
Stephen Harrop	Vodafone	Contributor
Tassos Sarantis	CityFibre	Reviewer
Vance Shipley	Sigscale	Contributor